

Agenda

- 1) Model network as weighted graph
- 2) BFS for hop-count shortest path
- 3) DFS for topology & cycle detection
- 4) Dijkstra's for least cost path routing.

Network as Graphs

A graph $G = (V, E)$ where:

V (vertices) : Routers / network nodes

E (Edges) : Network links between nodes

Edge Weight : link cost (latency, bandwidth, hop-cost)



Adjacency list

graph = d

'Mumbai' : [('Delhi', 5), ('Hyderabad', 3)]

'Delhi' : [(M, 5), (H, 2), (C, 8)]

'Hyderabad' : [(M, 3), (D, 2), (B, 2)]

'Chennai' : [(D, 8), (B, 4)]

'Bangalore' : [(H, 2), (C, 4)]

}

Adjacency Matrix

		M	D	H	C	B
		0	1	2	3	4
M	0	0	5	3	0	0
D	1	5	0	2	8	0
H	2	3	2	0	0	2
C	3	0	8	0	0	4
B	4	0	0	2	4	0

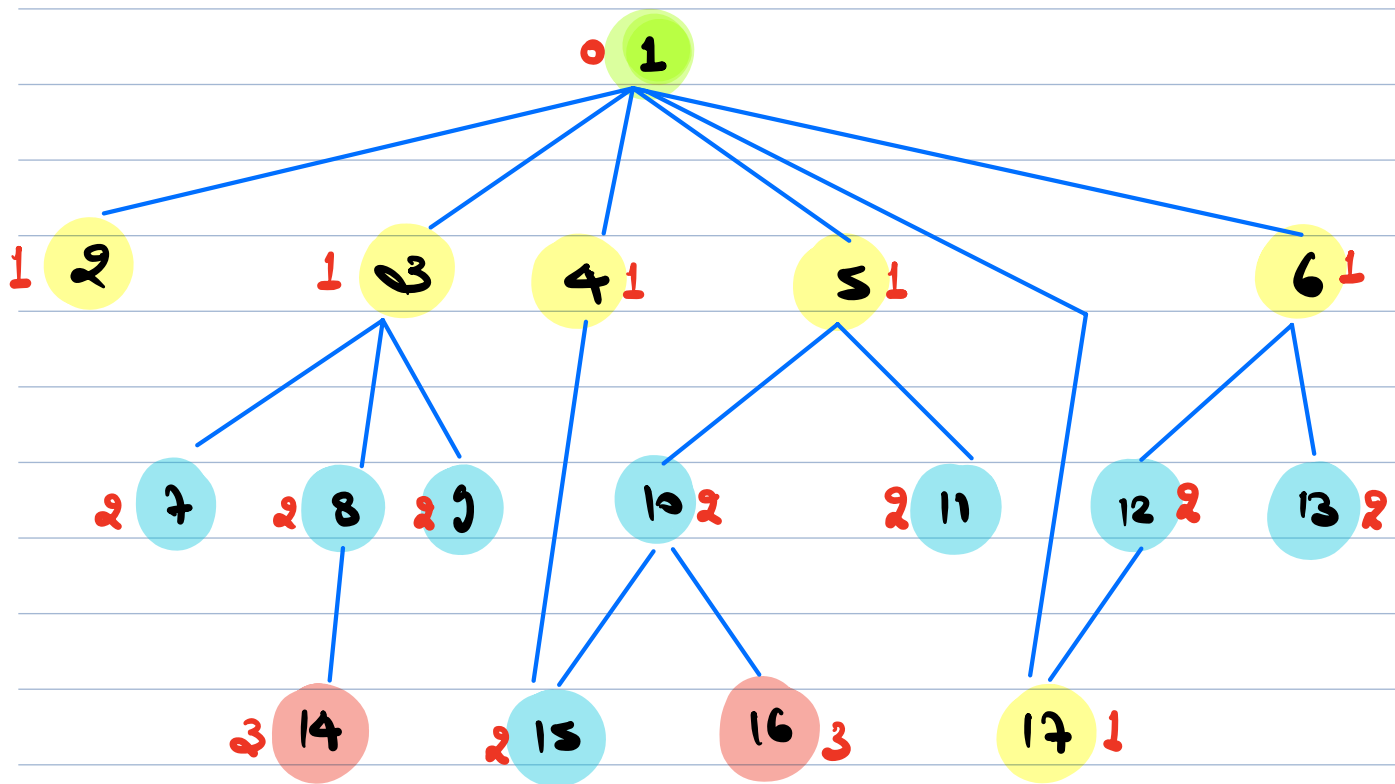
Mat[i][j]

0 (no path from i to j)

$W_{i \rightarrow j}$ (Weight of edge if path exists)

Graph Traversal

Min Hop Count Path from Source
to All Reachable Nodes



BFS (Breadth First Search)

Code

Nodes $\rightarrow [1, N]$

```
Queue <Node> q;
```

```
HashSet <Node> vst;
```

```
q.enqueue(src);
```

```
Map <Node, int> ans;
```

```
ans.insert(src, 0);
```

```
vst.insert(src);
```

```
while (!q.isEmpty()) {
```

```
    Node x = q.dequeue();
```

```
    int hc = ans.get(x);
```

```
    for (all nodes u connected to x) {
```

```
        if (!vst.contains(u)) {
```

```
            vst.insert(u);
```

```
            ans.insert(u, hc+1);
```

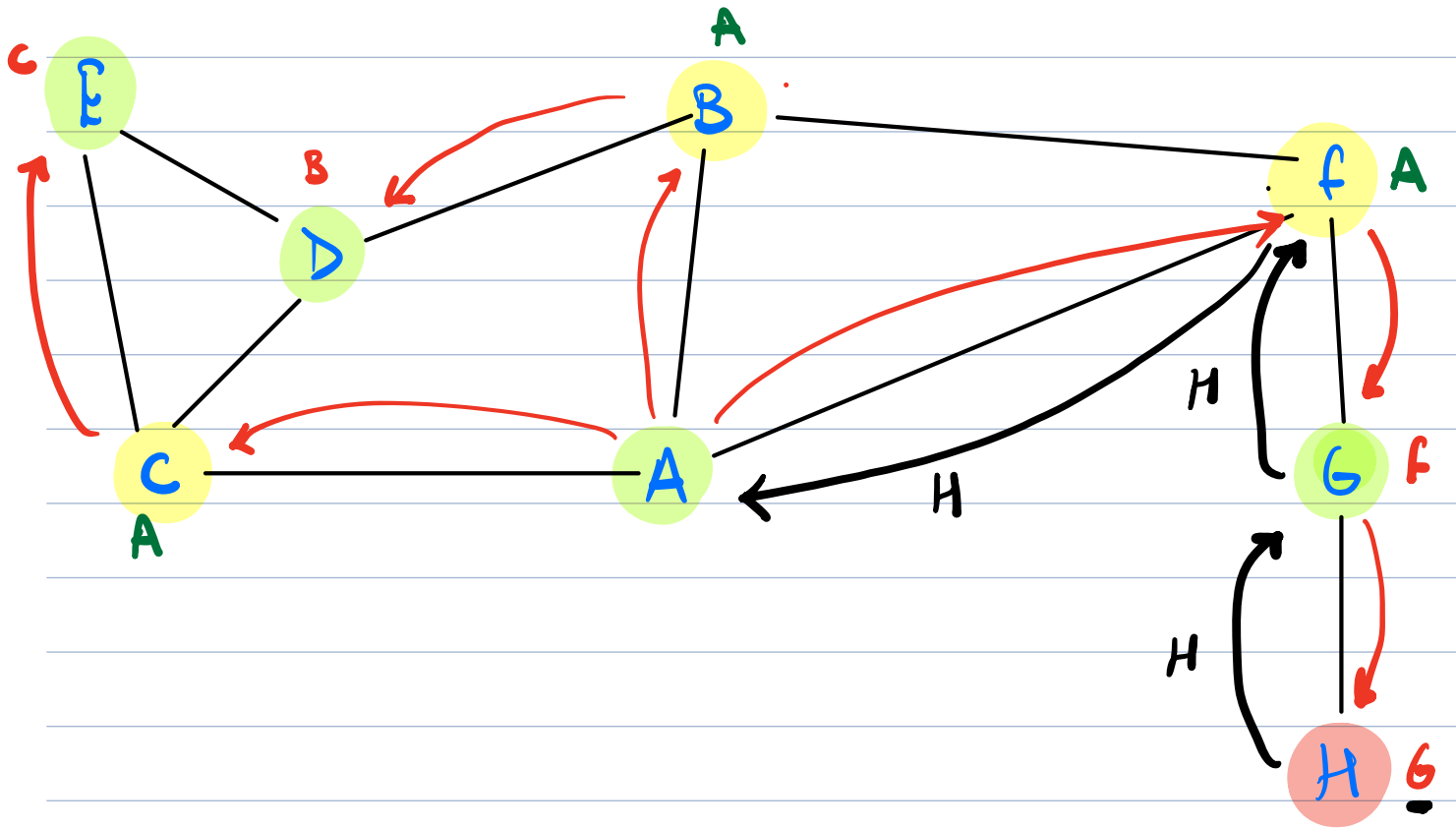
```
            q.enqueue(u);
```

```
        }
```

```
    }
```

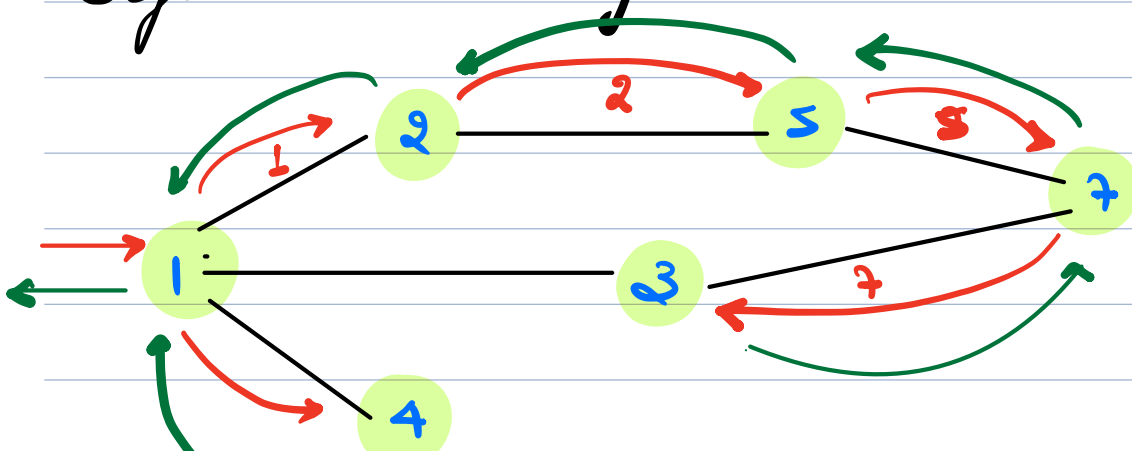
```
}
```

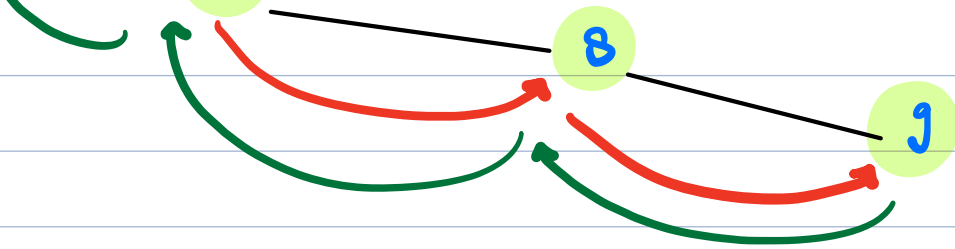
BFS With Path Tracing



Dfs (Depth first Search)

Explores as deep as possible along one branch before backtracking.





Code

```
HashSet<Node> vst;
```

```
void dfs (src) {
```

```
    vst.insert (src);
```

```
    for (all Node u connected to src) {
```

```
        if (! vst.contains (u)) {
```

```
            dfs (u);
```

```
        }
```

```
    }
```

```
}
```

Code

```
HashSet<Node> vst;
```

NULL (for src)



```
bool isCyclePresent (src, parent) {
```

```
    vst.add (src)
```

for (all Node u connected to src) {

if (vst.contains(u) && (u != parent)) {
return true;

} else if (!vst.contains(u)) {

if (isCyclePresent(u)) {
return true;

}

}

}

return false;

}